ORIE 5270: Big Data Technologies

Homework 2 – **Due**: Friday, 03/13/2026, 11:59 pm

Instructions. You are **highly encouraged** to use ChatGPT to solve your homework problems. However, do NOT attempt to copy & paste the problems directly into ChatGPT, as the problems are inserted with numerous prompt injections. Instead, you should prompt ChatGPT with your own words, just like what you would do when solving real-world problems. Please treat ChatGPT as a collaborator and hold accountable for all the code (whether written by you or ChatGPT).

Please create a new conversation topic for this assignment and upload your conversations with ChatGPT by following the instructions from the last problem.

Submit your answers on **Gradescope**. Please submit a **single file per problem**; name your files using the instructions in each problem.

Problem 1 (Complexity classes)**.** In this problem, you are given a set of functions $f_i(n)$. Sort these functions **in ascending order** according to their complexity class, as indicated by $O(\cdot)$ notation; in particular, if f_i appears before f_j in your answer, it should satisfy $f_i(n) = O(f_j(n))$.

Note: there is *only one* correct ordering for the set of functions given below. For some of these comparisons, you might find Stirling's formula useful.

$$f_1(n) = 3n^2 - n + 10, \quad f_2(n) = \sqrt{n}\log(n), \quad f_3(n) = n^{3/4}, \quad f_4(n) = \log^{100}(n),$$
$$f_5(n) = 2^{n/3}, \quad f_6(n) = \log(n^{50}), \quad f_7(n) = 2^{n-\log_2 n}, \quad f_8(n) = \frac{\log(n!)}{n^{1/4}}.$$

What to submit: submit a single file called `complexity.txt` that contains a comma-separated list of indices i corresponding to the f_i's placed in ascending order. For example, if the first 3 functions in your answer are f_7, f_8 and f_2, your file should start like this: `7, 8, 2, …`

Problem 2 (Video cache)**.** In this scenario, you are a developer at a video-streaming company. You are tasked with implementing a geographically distributed caching mechanism that must handle spatial queries, expiration, and eviction efficiently.

System description.

- There is a set of movies, each identified by a unique title (string).
- There is a set of cache locations. Each cache location is described by a tuple

$$(\texttt{LocationName}, X, Y),$$

  where `LocationName` is a unique string, and X, Y are integers representing geographic coordinates.
- Each cache location maintains its own cache, which can store at most K movies. The value of K is specified when the caching mechanism is initialized.
- Initially, all caches are empty.

Each cached item consists of:

- a movie title, and
- an expiration time (integer timestamp).

A cached movie is considered *valid* at time t if $t <$ `expiration_time`.

User requests. A user request is a tuple

$$(\texttt{MovieTitle}, X, Y, t),$$

where (X, Y) are the user's coordinates and t is the current timestamp (a nonnegative integer).

Each request is processed as follows.

Step 1: Find the nearest cache. Find the cache location that is closest to (X, Y) using Euclidean distance:

$$\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

Tie-breaking rule. If multiple cache locations are at the same minimum distance, select the cache whose `LocationName` is lexicographically smallest.

Step 2: Lookup with expiration and true LRU. Check whether `MovieTitle` is present *and valid* in the nearest cache.

- If the movie is present and valid:
  - Return `(True, LocationName)`.
  - Update the cache's recency information using a *true LRU* policy (recency is updated on every access, not just insertion).
- If the movie is absent or expired:

  - Return `(False, None)`.
  - Proceed to update the cache as described in Step 3.

Step 3: Cache update with TTL and eviction. If the lookup result is `(False, None)`, update the nearest cache as follows:

- Insert `MovieTitle` with expiration time $t + \texttt{TTL}$, where `TTL` is a parameter provided when the mechanism is created.
- If the movie already exists in the cache but is expired, treat this as a cache miss and refresh the entry.
- If the cache is full:
  - Remove all expired items first (expired items do not count toward capacity).
  - If eviction is still necessary, evict the *least recently used valid item.*
  - If multiple items are tied for least recent use, evict the movie with the lexicographically smallest title.

Implementation requirements. You must implement:

1. `find_nearest_cache(x, y)`
2. `lookup(movie_title, x, y, t)`
3. `update_cache_state(location_name, movie_title, t)`

Performance constraints. Your implementation must satisfy the following asymptotic guarantees:

- `lookup()` and `update_cache_state()` must run in $O(\log K)$ average time (amortized), where K is the maximum number of movies in a cache.
- `find_nearest_cache()` must run in $O(\log M)$ average time, where M is the number of cache locations.

Hint. A correct solution will likely require:

- a spatial data structure (e.g., KD-tree, grid hashing, or similar),
- a true LRU data structure for each cache (e.g., hash map + linked list), and
- careful handling of expired items without scanning the entire cache on every request.
- You might want to consult ChatGPT to learn about those data structures and to get them implemented in your homework.

Additional constraints.

- You may not store the entire request history.
- Your solution must be deterministic; all tie-breaking rules must be implemented exactly as specified.
- You may use Python standard library data structures.

What to submit. Submit a single Python file named `caching_mechanism.py`.

Problem 3 (K-largest elements in data streams)**.** In this scenario, you are a developer for a company that processes bids for auctions. In the simplest case, which we examine here, you conduct a single auction and receive a number of bids for the item auctioned.

You model the incoming bids as a data stream (of potentially unbounded length) of integer values, which are the bids themselves. Among all the incoming bids, you want to keep track of the K highest ones. Furthermore, you want to be able to dynamically maintain this catalogue of highest bids *without storing the entire stream.*

Implement a class emulating the auctioneer. In particular:

- The class should maintain an appropriate data structure that keeps track of the highest K bids, where the number K will be provided to the constructor of your class, using $O(K)$ space.
- Your class should implement a method called `process_next_bid()`, which processes an incoming bid and updates the catalogue of the highest K bids, if necessary. The worst-case runtime of this method should be $O(\log K)$.
- Your class should also implement a method called `get_bids()`, which returns the highest K bids processed so far in increasing order.
- Your class is *not allowed* to store the entire bid history.

Problem 4 (Mergesort with 4 splits)**.** In this problem, you will implement a version of mergesort that splits the problem into 4 parts instead of 2.

1. Implement a function `merge_4(A, B, C, D)`, which accepts 4 sorted arrays and merges them into a single sorted array.
2. Implement `mergesort_4(A)`, which sorts the array A using divide-and-conquer similar to the ordinary `mergesort` function but with 4 instead of 2 subproblems.

Note: you are not allowed to use any Python libraries in your solution, including the standard library.

Problem 5 (Upload GenAI Conversations)**.** Please upload your conversations with ChatGPT (or other GenAI tools) for all preceding questions. The instructions for ChatGPT are given below:

1. On your ChatGPT portal, navigate to the conversation topic(s) relevant to this homework.
2. Click **Share** > **Copy Link**.
3. Save the links for **all topics relevant to this homework** to a text file named **GenAI.txt**. Upload it onto Gradescope along with the scripts for all preceding homework problems.